

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МО ЭВМ

Отчет
по лабораторной работе №1
по дисциплине «Введение в машинное обучение»
Тема: Изучение и предобработка данных.

Студентка гр. 3384

Копасова К. А.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель работы: Изучить предобработку данных с использованием python библиотек Pandas, Seaborn и NumPy.

Задание: Вариант 5 - lab1_var5.csv.

1. Изучение набора данных с использованием Pandas и Seaborn:

1.1. Загрузить данные из файла как Pandas DataFrame

1.2. Вызвав у датафрейма метод `head`, проверить корректность загруженных данных.

1.3. Вызвав у датафрейма метод `describe`, получить характеристики. Опишите полученный результат.

1.4. Видоизмените полученный датафрейм таким образом, чтобы метка классов были следующими: `Class1`, `Class2` и т.д.. Сохраните полученный датафрейм в отдельный файл формата csv.

1.5. Визуально оцените набор данных, построив изображение, содержащее графики ядерной оценки плотности каждого признака (кроме признака названия/ номера класса), диаграмму рассеяния и двумерную ядерную оценку плотности для каждого признака. Наблюдения разных классов должны быть выделены отдельным цветом (рекомендуемая палитра `'tab10'` или `'Set1'`). Пример построения: https://seaborn.pydata.org/examples/pair_grid_with_kde.html . Опишите полученный график, что на нем изображено, какие выводы о данных можно сделать.

1.6. На одном изображении постройте гистограммы распределения для каждого признака (для построения нескольких диаграмм на одном изображении, необходимо создать subplot из `matplotlib`, и для каждой диаграммы задать параметр `ax`, указав нужную ячейку. `subplot` возвращает два параметра: саму фигуру с изображением и список ячеек. Например, изображение с 4 ячейками записанных в ряд: `fig, axs = plt.subplots(1,4)`. Указание ячейки в параметре диаграммы делается следующим образом: `ax=axs[0]`). Затем последовательно модифицируйте изображение.

1.6.1. Постройте гистограммы для разного количества столбцов: 5,10,15,20,30. Выберите на ваш взгляд такое количество столбцов, который лучше всего описывает форму распределения признаков.

1.6.2. Сделайте на каждой гистограмме разделение по цвету согласно классу. Проведите это в двух режимах, когда гистограммы накладываются/суммируются и когда пересекаются. Далее используйте режим с пересечением.

1.7. Постройте гистограммы, чтобы вместо столбцов изображались ступеньки.

1.8. Добавьте на гистограммы график ядерной оценки плотности.

2. Изучение набора данных с использованием NumPy:

2.1. Загрузите данные из файла как массив NumPy.

2.2. Выведите первые 10 наблюдений набора данных.

2.3. Рассчитайте характеристики полученные методом describe в п. 1.3 с использованием методов NumPy. Обращайте внимание на то, где оценка смещенная, а где нет.

3. Преобразование данных:

3.1. Получите из датафрейма из п. 1.4 столбец с названием классов. Используя LabelEncoder и OneHotEncoder получите различные способы кодирования меток класса. В чем различия полученных кодировок?

3.2. Для датафрейма из п. 1.4, получите все столбцы признаков (столбцы не содержащие метки классов). Преобразуйте полученные столбцы в массив NumPy.

3.3. Для массива NumPy из п. 3.2 примените StandardScaler, MinMaxScaler, MaxAbsScaler и RobustScaler. Для каждого из результатов постройте гистограммы по каждому признаку без разделения по классам. В чем различия между такими преобразованиями данных? Какая трансформация подходит лучше всего под ваш датасет?

4. Понижение размерности:

4.1. Для набора данных примените понижение размерности, используя PCA и TSNE из Sklearn. Для PCA оцените необходимое количество компонент для сохранения информации ~90%. Для каждого из результатов постройте диаграмму рассеяния с выделением разным цветом наблюдений разных классов. Объясните полученные результаты.

Выполнение работы

1. Изучение набора данных с использованием Pandas и Seaborn:

1.1. Загрузка данных из файла как Pandas DataFrame

Для загрузки данных использована функция `read_csv()` библиотеки Pandas:

```
import pandas
data_pandas = pandas.read_csv('lab1_var5.csv')
```

Функция `read_csv()` автоматически определяет разделитель (запятая), считывает заголовки столбцов из первой строки файла и создает DataFrame - табличную структуру данных с индексацией строк.

1.2. Проверка корректности загруженных данных методом `head()`

```
print(data_pandas.head())
```

Метод `head()` вывел первые 5 строк датасета (см. рис. 1.1).

	Unnamed: 0	Var1	Var2	Var3	Var4	Var5	Label
0	0	-13.631926	12.573362	6.088984	-10.642815	-8.039519	2
1	1	-12.918539	8.928075	6.252582	-9.658388	-12.872628	0
2	2	-12.817660	11.985211	7.387556	-10.210012	-9.624412	2
3	3	-12.668943	7.622253	9.482878	-9.135697	-12.057752	0
4	4	-12.292029	7.903125	8.811845	-9.402879	-11.757409	2

Рисунок 1.1 - Проверка корректности загруженных данных.

Вывод подтвердил корректность загрузки: присутствуют 7 столбцов (Unnamed: 0, Var1-Var5, Label), данные числовые.

1.3. Получение характеристик методом `describe()`

```
print(data_pandas.describe())
```

Метод `describe()` содержит следующие характеристики: `count` (количество записей), `mean` (среднее значение), `std` (стандартное отклонение), `min` (минимальное значение), 25% (первый квартиль), 50% (медиана), 75% (третий квартиль) и `max` (максимальное значение). Результат работы метода показан на рис. 1.2.

	Unnamed: 0	Var1	Var2	Var3	Var4	Var5
count	700.00000	700.00000	700.00000	700.00000	700.00000	700.00000
mean	349.50000	-10.620139	10.705185	9.358156	-9.944875	-10.643754
std	202.21688	2.079660	2.121356	2.159222	0.568040	2.217638
min	0.00000	-14.132361	5.489138	4.676009	-11.485496	-15.839444
25%	174.75000	-12.156922	8.611177	7.715782	-10.390090	-12.322772
50%	349.50000	-11.437105	11.411947	8.823989	-9.899319	-11.097444
75%	524.25000	-8.929502	12.365377	11.184827	-9.536641	-8.892299
max	699.00000	-5.082175	15.402761	15.710911	-7.855822	-4.807697

	Label
count	700.00000
mean	1.027143
std	0.829089
min	0.000000
25%	0.000000
50%	1.000000
75%	2.000000
max	2.000000

Рисунок 1.2 - Использование метода describe().

Получаем следующий результат:

1) Датасет содержит 700 наблюдений, пропусков нет.

2) Столбец Unnamed: 0 содержит индексы объектов (0–699), не является признаком.

3) Признаки Var1-Var5 имеют разные диапазоны значений и стандартные отклонения: наименьший разброс у Var4 (std=0.57), наибольший - у Var5 (std=2.22).

4) Целевая переменная Label принимает значения 0, 1, 2, что указывает на задачу многоклассовой классификации с тремя классами.

5) Распределение классов неравномерное: медиана равна 1, что говорит о преобладании второго класса.

1.4. Изменение и сохранение датафрейма

```
data_pandas['Label'] = 'Class' + (data_pandas['Label'].astype(int) + 1).astype(str)
data_pandas.to_csv('lab1_var5_with_class.csv', index=False)
```

Числовые метрики (0, 1, 2) преобразованы в строковые значения Class1, Class2, Class3. С помощью метода to_csv() обновленные данные были загружены в файл lab1_var5_with_class.csv (параметр index=False предотвращает сохранение индекса в файл).

1.5. Визуальная оценка данных: KDE, scatter plot, pair plot

Для визуального анализа взаимосвязей между признаками использована функция PairGrid() библиотеки Seaborn:

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt

var_cols = [col for col in data_pandas.columns if col != 'Label' and col !=
'Unnamed: 0']

g = sns.PairGrid(data_pandas, vars=var_cols, hue='Label', palette='tab10')
g.map_upper(sns.scatterplot, s=15)
g.map_lower(sns.kdeplot)
g.map_diag(sns.kdeplot, lw=2)
g.add_legend(title='Класс', fontsize=9)
plt.suptitle('PairGrid: scatter, KDE по диагонали и снизу', y=1.02)
plt.show()
```

Для визуального анализа использован метод PairGrid(), который строит матрицу графиков для признаков Var1–Var5. Параметр hue='Label' позволяет отображать классы разными цветами. В верхней части построены диаграммы рассеяния с помощью scatterplot, в нижней - двумерные KDE-графики, а на диагонали - одномерные KDE-распределения признаков. Общий график позволяет оценить распределение каждого признака и взаимосвязи между ними. На рис. 1.5 представлен результат выполнения кода.

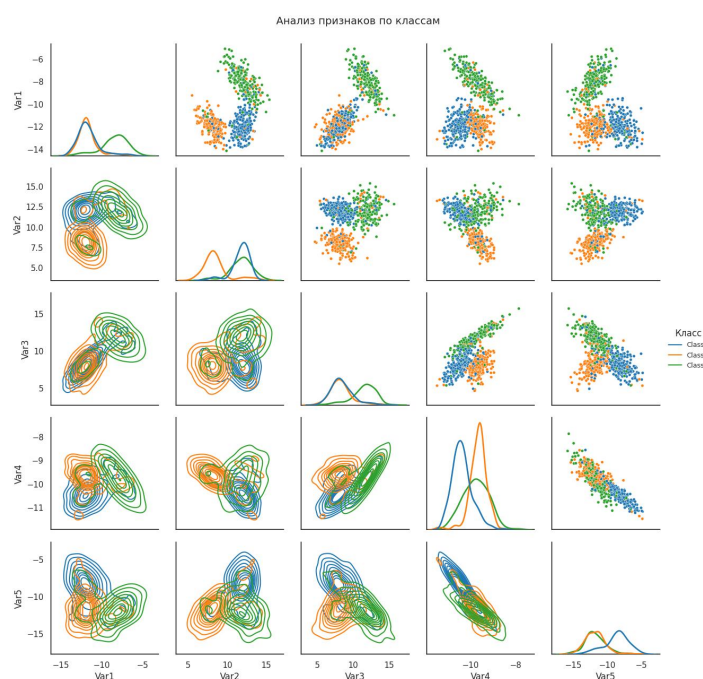


Рисунок 1.5 - PairGrid с scatter plot (верхний треугольник), KDE plot (нижний треугольник) и KDE по диагонали.

На диагонали видно, что распределения признаков для разных классов часто перекрываются, но по некоторым признакам (Var1, Var2, Var5) классы 1 и 2 имеют заметные различия в форме кривых. В верхнем треугольнике диаграммы

рассеяния показывают, что пары признаков не образуют четких линейных границ между классами. Нижние KDE-контуры показывают, что плотность распределения наблюдений разных классов частично накладывается, особенно для классов 2 и 3 по признакам Var3 и Var4.

1.6. Построение гистограмм распределения признаков

```
fig, axs = plt.subplots(1, len(var_cols), figsize=(5*len(var_cols), 4))

for i, feature in enumerate(var_cols):
    sns.histplot(data_pandas[feature], ax=axs[i])
    axs[i].set_title(feature)

plt.tight_layout()
```

Для построения гистограмм использован `sns.histplot()` в цикле по всем признакам. Параметр `ax` позволяет размещать каждый график в отдельной ячейке, созданной через `plt.subplots()`. На рис. 1.6 представлен результат выполнения кода.

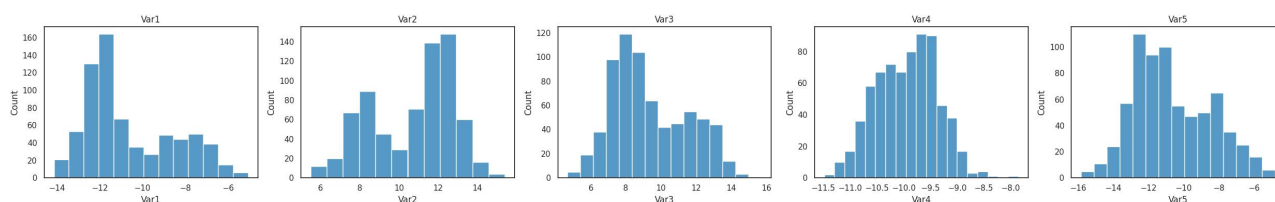


Рисунок 1.6 - Гистограммы распределения признаков.

Гистограммы показывают общую форму распределения каждого признака. Видно, что некоторые признаки (например, Var1) имеют смещенные распределения для разных классов, тогда как другие (например, Var4) почти одинаковы для всех классов.

1.6.1. Выбор оптимального количества бинов

Выясним экспериментальным путем, какое оптимальное количество столбцов будет лучше оценивать распределения для каждого признака.

```
bins_list = [5, 10, 15, 20, 30]

for bins in bins_list:
    fig, axs = plt.subplots(1, len(var_cols), figsize=(5*len(var_cols), 4))

    for i, feature in enumerate(var_cols):
        sns.histplot(data_pandas[feature], bins=bins, ax=axs[i])
        axs[i].set_title(f"{feature}, bins={bins}")
```



```
plt.tight_layout()
plt.show()
```

В цикле перебираются значения количества бинов: 5, 10, 15, 20, 30. Для каждого значения строится новый набор гистограмм. Это позволяет визуально сравнить, при каком количестве столбцов форма распределения видна наиболее четко, но без излишнего шума. На рис. 1.6.1.1, 1.6.1.2, 1.6.1.3, 1.6.1.4, 1.6.1.5 представлены результаты выполнения кода.

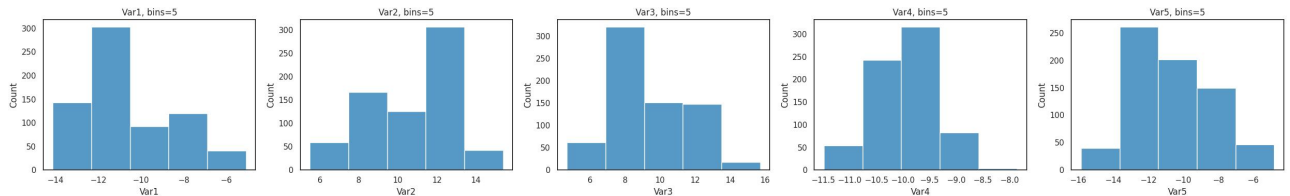


Рисунок 1.6.1.1 - Гистограммы распределения признаков при bins=5.

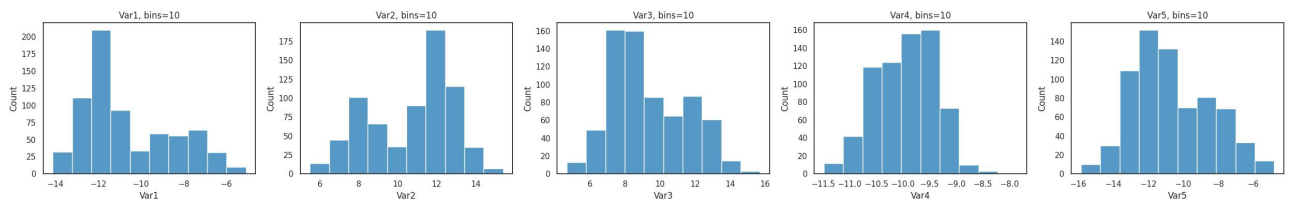


Рисунок 1.6.1.2 - Гистограммы распределения признаков при bins=10.

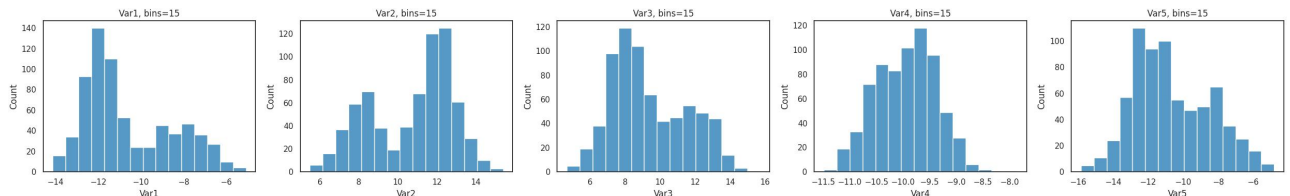


Рисунок 1.6.1.3 - Гистограммы распределения признаков при bins=15.

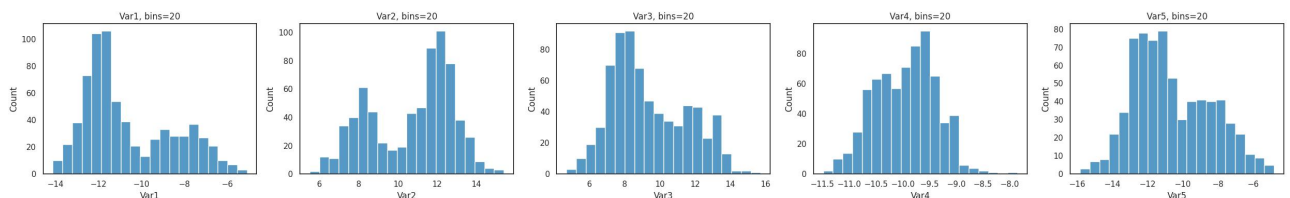


Рисунок 1.6.1.4 - Гистограммы распределения признаков при bins=20.

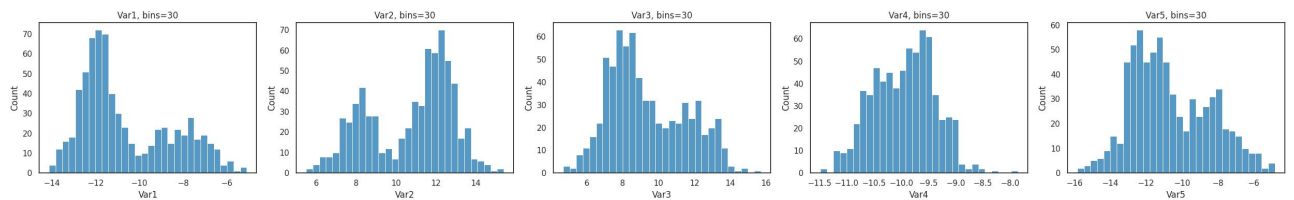


Рисунок 1.6.1.5 - Гистограммы распределения признаков при bins=30.

Оптимальным количеством бинов для данного набора оказалось 15: при этом форму распределения видно достаточно четко, но без излишней фрагментации.

1.6.2. Разделение гистограмм по классам

Суммирование гистограмм:

```
target_col = data_pandas.columns[-1]
fig, axs = plt.subplots(1, len(var_cols), figsize=(20, 4))

for i, col in enumerate(var_cols):
    sns.histplot(
        data=data_pandas,
        x=col,
        hue=target_col,
        bins=15,
        multiple="stack",
        ax=axs[i]
    )
    axs[i].set_title(col)

plt.tight_layout()
plt.show()
```

Результат работы представлен на рис. 1.6.2.1.

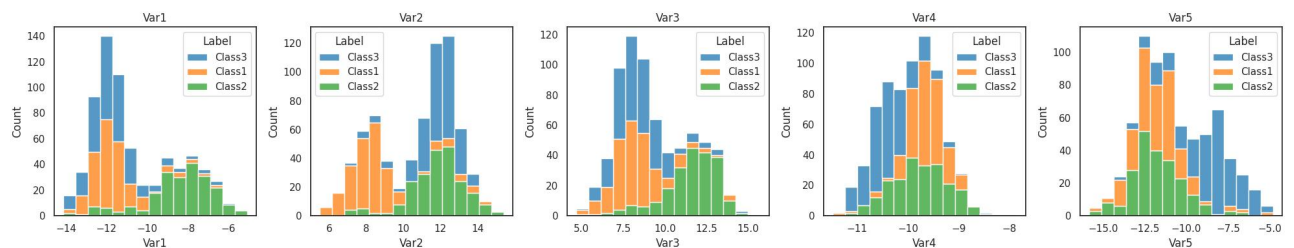


Рисунок 1.6.2.1 - Суммирование гистограмм.

Пересечение гистограмм:

```
fig, axs = plt.subplots(1, len(var_cols), figsize=(20, 4))
for i, col in enumerate(var_cols):
    sns.histplot(
        data=data_pandas,
        x=col,
        hue=target_col,
```

```

        bins=15,
        multiple="layer",
        ax=axis[i]
    )
    axis[i].set_title(col)

plt.tight_layout()
plt.show()

```

Результат работы представлен на рис. 1.6.2.2.

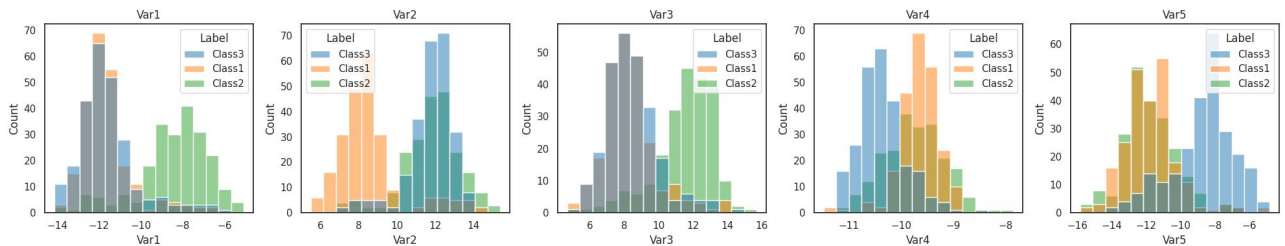


Рисунок 1.6.2.2 - Пересечение гистограмм.

Параметр `hue=target_col` в `histplot()` разделяет данные по классам и раскрашивает их разными цветами. Параметр `multiple` управляет способом отображения: "stack" складывает гистограммы друг на друга (суммирование), "layer" накладывает их с прозрачностью (пересечение).

В режиме пересечения видно, что для признаков Var1, Var2, Var5 распределения классов заметно смещены относительно друг друга - это делает их полезными для классификации. Для Var3 и Var4 кривые практически совпадают, следовательно, эти признаки несут меньше информации о принадлежности к классу.

1.7. Гистограммы в виде ступенек

```

fig, axis = plt.subplots(1, len(var_cols), figsize=(5*len(var_cols), 4))

for i, feature in enumerate(var_cols):
    sns.histplot(data_pandas[feature], ax=axis[i], element='step')
    axis[i].set_title(feature)

plt.tight_layout()

```

Параметр `element='step'` в `histplot()` заменяет обычные столбцы на ступенчатую линию. Результат работы представлен на рис. 1.7.

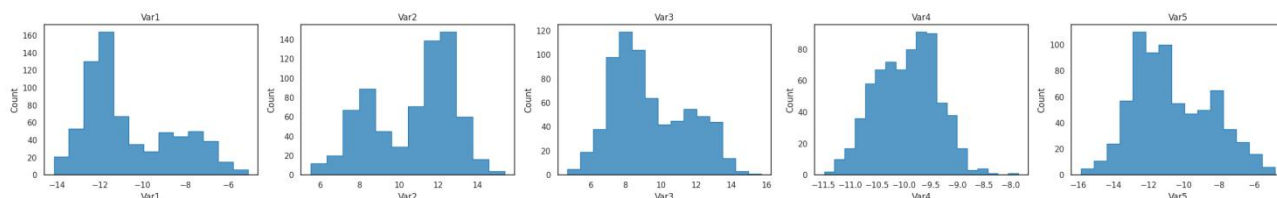


Рисунок 1.7 - Гистограммы распределения признаков в виде ступенек.

Ступенчатые гистограммы сохраняют всю информацию о распределении, улучшают читаемость графиков и помогают точнее оценивать различия между распределениями.

1.8. Добавление KDE на гистограммы

```
fig, axs = plt.subplots(1, len(var_cols), figsize=(5*len(var_cols), 4))

for i, feature in enumerate(var_cols):
    sns.histplot(data_pandas[feature], ax=axs[i], element='step', kde=True)
    axs[i].set_title(feature)

plt.tight_layout()
```

Параметр `kde=True` добавляет поверх гистограммы сглаженную кривую ядерной оценки плотности. Результат работы представлен на рис. 1.8.

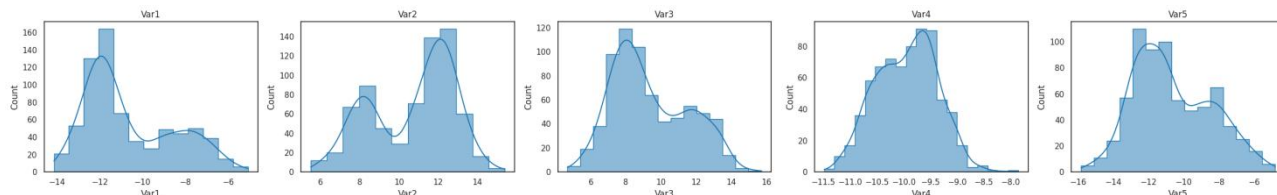


Рисунок 1.8 - Гистограммы распределения признаков с KDE.

Наложение KDE подтверждает выводы, сделанные по гистограммам: для одних признаков распределения классов четко разделены, для других - сильно перекрываются. При этом KDE сглаживает шум, делая выводы более надежными.

2. Изучение набора данных с использованием NumPy

2.1. Загрузка данных как массив NumPy

```
import numpy as np
data_numpy = np.genfromtxt('lab1_var5.csv', delimiter=',', skip_header=1)
```

Функция `np.genfromtxt()` читает файл и возвращает числовой массив `numpy`. Параметр `delimiter=''` указывает разделитель, а `skip_header=1` пропускает строку с заголовками, чтобы в массив попали только данные.

2.2. Вывод первых 10 наблюдений

```
print(data_numpy.shape)
print(data_numpy[:10])
```

Срез `data_numpy[:10]` возвращает первые десять строк массива. Дополнительно выводится `shape`, чтобы проверить размерность. Результат работы представлен на рис. 2.2.

```
(700, 7)
[[ 0. -13.63192607 12.57336243 6.08898409 -10.64281463
  -8.03951862 2. ]
 [ 1. -12.91853937 8.92807473 6.25258214 -9.65838801
 -12.87262786 0. ]
 [ 2. -12.81765988 11.98521084 7.38755638 -10.21001156
  -9.62441238 2. ]
 [ 3. -12.66894276 7.62225263 9.48287819 -9.13569723
 -12.05775214 0. ]
 [ 4. -12.29202943 7.90312482 8.81184465 -9.40287889
 -11.75740912 2. ]
 [ 5. -11.57372718 7.46294643 8.74773965 -9.48621864
 -11.91308043 0. ]
 [ 6. -12.92296671 8.60295002 7.18311569 -9.57273799
 -12.2179839 1. ]
 [ 7. -10.66298955 12.31061688 8.77479529 -10.56794001
  -8.28066065 2. ]
 [ 8. -7.96542875 10.38700033 13.46974198 -9.34933469
 -11.9041834 1. ]
 [ 9. -11.66917995 7.07800583 8.17603745 -9.41324732
 -12.73381675 0. ]]
```

Рисунок 2.2 - Проверка набора данных, загруженных через `numpy`.

Значения в первых десяти строках совпадают с теми, что были при загрузке через `Pandas`. Размерность `(700, 7)` подтверждает, что никакие данные не потерялись.

2.3. Расчет характеристик методами `NumPy`

```
def describe_numpy(data_numpy):
    desc = {}
    desc['count'] = data_numpy.shape[0]
    desc['mean'] = np.mean(data_numpy, axis=0)
    desc['std'] = np.std(data_numpy, axis=0, ddof=1) # ddof=1 - несмещенная
оценка, делит на (n-1)
    desc['min'] = np.min(data_numpy, axis=0)
    desc['25%'] = np.percentile(data_numpy, 25, axis=0)
    desc['50%'] = np.median(data_numpy, axis=0)
    desc['75%'] = np.percentile(data_numpy, 75, axis=0)
    desc['max'] = np.max(data_numpy, axis=0)

    return desc

desc = describe_numpy(data_numpy)
```

```
for key, value in desc.items():
    print(f"{key}: {value}")
```

Функция `describe_numpy()` вручную вычисляет те же статистики, что и метод `describe()` в Pandas, но с использованием функций `numpy`. Для стандартного отклонения указан параметр `ddof=1`, он дает несмещенную оценку (деление на $n-1$), по умолчанию `numpy` использует смещенную оценку (деление на n). Результат работы представлен на рис. 2.3.

```
count: 700
mean: [349.5      -10.62013921  10.70518523   9.35815569  -9.94487518
 -10.64375443   1.02714286]
std: [202.21688027   2.07965997   2.12135579   2.15922245   0.56804037
    2.21763789   0.82908872]
min: [ 0.      -14.13236132   5.48913825   4.67600885  -11.48549601
 -15.83944396   0.      ]
25%: [174.75    -12.15692174   8.61117663   7.71578153  -10.39008959
 -12.32277214   0.      ]
50%: [349.5     -11.43710478  11.41194685   8.8239894   -9.89931899
 -11.09744383   1.      ]
75%: [524.25     -8.92950196  12.36537688  11.18482685  -9.53664124
 -8.89229884   2.      ]
max: [699.      -5.0821751   15.40276093  15.71091131  -7.8558223
 -4.80769701   2.      ]
```

Рисунок 2.3 - Рассчитанные характеристики набора данных.

Полученные значения полностью совпадают с результатами из Pandas, что подтверждает корректность расчетов.

3. Преобразование данных

3.1. Кодирование меток класса

Загрузим данные столбца `label` с классами для проверки работы `LabelEncoder` и `OneHotEncoder`.

```
data_pandas_with_class = pandas.read_csv('lab1_var5_with_class.csv')
labels = data_pandas_with_class['Label']
print(labels[:5])
```

Результат работы представлен на рис. 3.1.1.

```
0    Class3
1    Class1
2    Class3
3    Class1
4    Class3
Name: Label, dtype: object
```

Рисунок 3.1.1 - Проверка загруженных лейблов.

Данные успешно загрузились.

Применим LabelEncoder к массиву labels.

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
label_encoder = LabelEncoder()
data_label_encoder = label_encoder.fit_transform(labels)
print(data_label_encoder[:10])
```

LabelEncoder преобразует строковые метки Class1, Class2, Class3 в целые числа 0, 1, 2. Это экономит память и подходит для целевой переменной в задачах классификации. Результат работы представлен на рис. 3.1.2.

[2 0 2 0 2 0 1 2 1 0]

Рисунок 3.1.2 - Результат работы LabelEncoder.

Применим OneHotEncoder к массиву labels.

```
one_hot_encoder = OneHotEncoder(sparse_output=False)
labels_reshaped = labels.values.reshape(-1, 1)
print(labels_reshaped[:5])
data_one_hot_encoder = one_hot_encoder.fit_transform(labels_reshaped)
print(data_one_hot_encoder[:10])
```

OneHotEncoder создает отдельный бинарный столбец для каждого класса (например, [1,0,0] для Class1), что устраняет ложный порядок между классами, но увеличивает размерность данных. Параметр sparse_output=False возвращает обычный массив вместо разреженной матрицы. Результат работы представлен на рис. 3.1.3.

[['Class3']
['Class1']
['Class3']
['Class1']
['Class3']]
[[0. 0. 1.]
[1. 0. 0.]
[0. 0. 1.]
[1. 0. 0.]
[0. 0. 1.]
[1. 0. 0.]
[0. 1. 0.]
[0. 0. 1.]
[0. 1. 0.]
[1. 0. 0.]

Рисунок 3.1.3 - Результат работы OneHotEncoder.

LabelEncoder подходит, когда метки - это целевая переменная, так как большинство алгоритмов ожидают целочисленные классы. OneHotEncoder нужен, когда категориальные признаки подаются на вход модели: если оставить целые числа, модель может ошибочно интерпретировать их как порядковые.

3.2. Преобразование признаков в массив NumPy

```
data_only_var = data_pandas.drop(columns=['Label', 'Unnamed: 0'])
print(data_only_var.head(), '\n')
data_numpy_array = np.array(data_only_var)
print(data_numpy_array.shape)
print(data_numpy_array[:5])
```

Сначала из DataFrame удаляются столбцы Label и Unnamed: 0, так как они не являются признаками. Оставшиеся столбцы Var1–Var5 преобразуются в массив numpy методом np.array(). Результат работы представлен на рис. 3.2.

	Var1	Var2	Var3	Var4	Var5
0	-13.631926	12.573362	6.088984	-10.642815	-8.039519
1	-12.918539	8.928075	6.252582	-9.658388	-12.872628
2	-12.817660	11.985211	7.387556	-10.210012	-9.624412
3	-12.668943	7.622253	9.482878	-9.135697	-12.057752
4	-12.292029	7.903125	8.811845	-9.402879	-11.757409

(700, 5)
[[-13.63192607 12.57336243 6.08898409 -10.64281463 -8.03951862]
[-12.91853937 8.92807473 6.25258214 -9.65838801 -12.87262786]
[-12.81765988 11.98521084 7.38755638 -10.21001156 -9.62441238]
[-12.66894276 7.62225263 9.48287819 -9.13569723 -12.05775214]
[-12.29202943 7.90312482 8.81184465 -9.40287889 -11.75740912]]

Рисунок 3.2 - Результат преобразования признаков в массив numpy.

Получен массив формы (700, 5), содержащий только признаки.

3.3. Масштабирование признаков

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, MaxAbsScaler, RobustScaler
```

```
scalers = {
    'StandardScaler': StandardScaler(),
    'MinMaxScaler': MinMaxScaler(),
    'MaxAbsScaler': MaxAbsScaler(),
    'RobustScaler': RobustScaler()
}

scaled_data = {}

for name, scaler in scalers.items():
    scaled_data[name] = scaler.fit_transform(data_numpy_array)
for name, data_scaled in scaled_data.items():
    n_features = data_scaled.shape[1]

    fig, axs = plt.subplots(1, n_features, figsize=(4*n_features, 4))
    fig.suptitle(name, fontsize=16)
```



```

for i in range(n_features):
    sns.histplot(data_scaled[:, i], bins=15, kde=True, ax=axes[i],
color='skyblue')
    axes[i].set_title(f'Var{i+1}')

plt.show()

```

Применяются четыре scaler из sklearn: StandardScaler центрирует данные и приводит к единичной дисперсии; MinMaxScaler сжимает значения в диапазон [0, 1]; MaxAbsScaler масштабирует относительно максимального по модулю значения; RobustScaler использует медиану и межквартильный размах, что делает его устойчивым к выбросам. Для каждого результата строятся гистограммы с KDE, чтобы визуальное сравнить форму распределений после трансформации. Результаты работы представлены на рис. 3.3.1, 3.3.2, 3.3.3, 3.3.4.

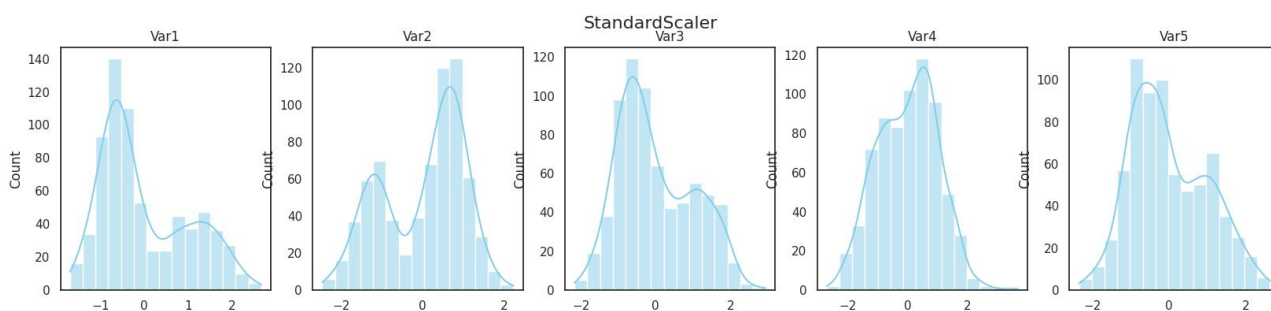


Рисунок 3.3.1 - Результат применения StandardScaler.

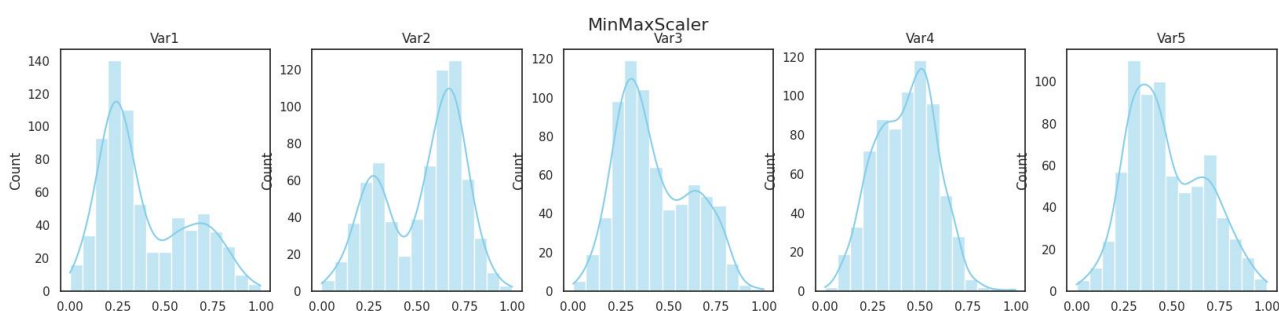


Рисунок 3.3.2 - Результат применения MinMaxScaler.

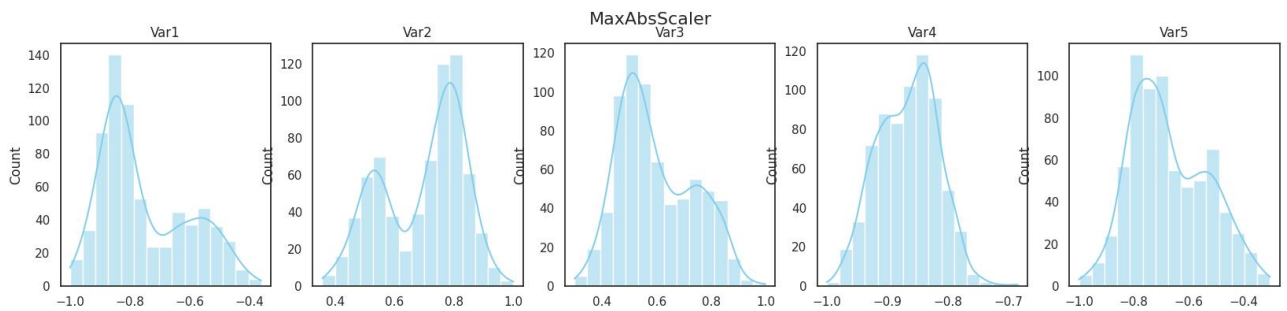


Рисунок 3.3.3 - Результат применения MaxAbsScaler.

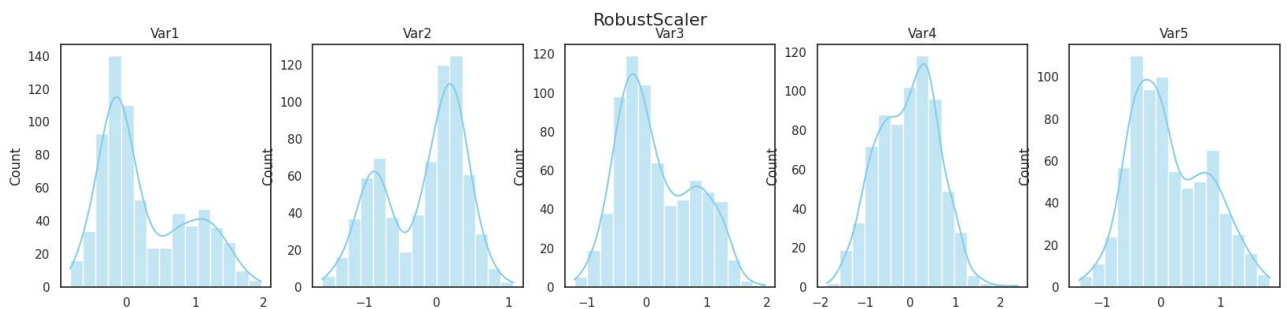


Рисунок 3.3.4 - Результат применения RobustScaler.

Сравним все scaler и проверим выбросы.

```
fig, axs = plt.subplots(1, len(var_cols), figsize=(5 * len(var_cols), 5))

for i in range(len(var_cols)):
    for name, data_scaled in scaled_data.items():
        sns.kdeplot(data_scaled[:, i], ax=axs[i], label=name, lw=2)
    axs[i].set_title(f'Var{i+1}')
    axs[i].legend(fontsize=8)

plt.suptitle('Сравнение scaler (ядерная оценка плотности)', fontsize=14)
plt.tight_layout()
plt.show()

fig, axs = plt.subplots(1, len(var_cols), figsize=(4 * len(var_cols), 4))
for i, col in enumerate(var_cols):
    sns.boxplot(y=data_pandas[col], ax=axs[i])
    axs[i].set_title(col)
plt.suptitle('Оценка выбросов', fontsize=14)
plt.tight_layout()
plt.show()
```

Результаты работы представлены на рис. 3.3.5 и рис. 3.3.6.

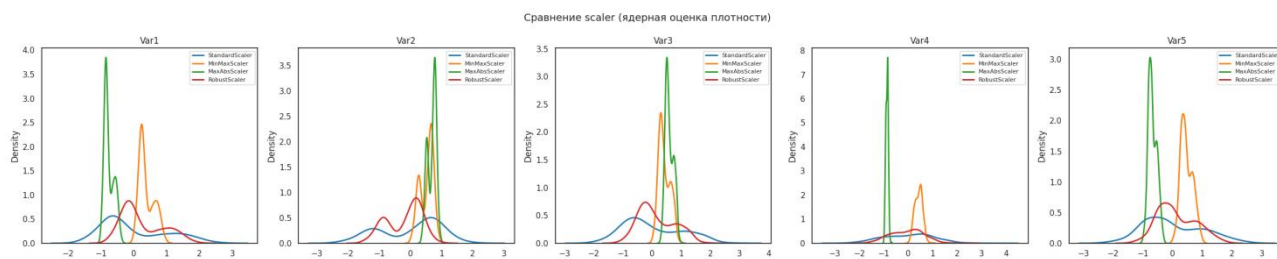


Рисунок 3.3.5 - Сравнение scaler.

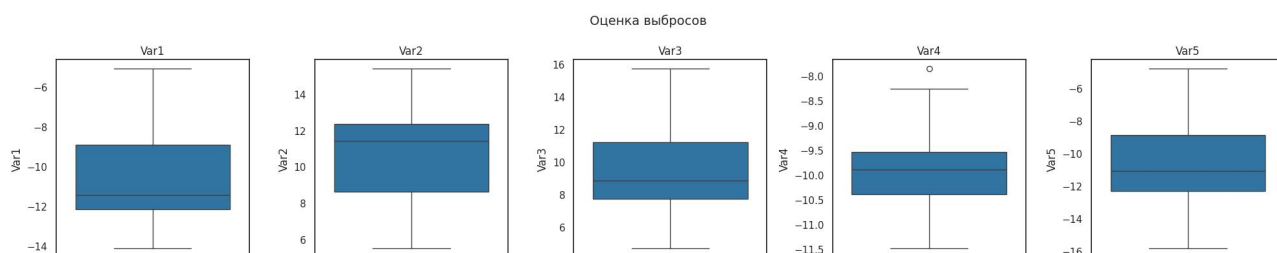


Рисунок 3.3.6 - Оценка выбросов.

Видим, что только у Var4 есть незначительный выброс, но в остальном все остальные данные имеют достаточно симметричные распределения.

Поскольку у данных выбросов практически нет, а распределения относительно симметричны, лучше всего подходит StandardScaler. RobustScaler дал бы аналогичный результат, но, поскольку его используют для устойчивости к выбросам, с этими данными он не актуален. MinMaxScaler и MaxAbsScaler слишком сжимают данные.

4. Понижение размерности

4.1. PCA и TSNE

Оценим для PCA необходимое количество компонент для сохранения 90% информации:

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

pca = PCA()
pca.fit(data_numpy_array)
cumulative_variance = np.cumsum(pca.explained_variance_ratio_)
n_components_90 = np.argmax(cumulative_variance >= 0.9) + 1
print("число компонент:", n_components_90)
```

Результат работы представлен на рис. 4.1.1.

Рисунок 4.1.1 - Результат вычисления числа компонент.

Построим диаграммы рассеяния для PCA и TSNE:

```
pca_2d = PCA(n_components=2)
data_pca = pca_2d.fit_transform(data_numpy_array)
plt.figure()
sns.scatterplot(x=data_pca[:,0], y=data_pca[:,1], hue=labels, palette='Set1')
plt.title("PCA")
plt.show()

tsne = TSNE(n_components=2)
data_tsne = tsne.fit_transform(data_numpy_array)
plt.figure()
sns.scatterplot(x=data_tsne[:,0], y=data_tsne[:,1], hue=labels, palette='Set1')
plt.title("TSNE")
plt.show()
```

Результаты работы представлены на рис. 4.1.2, 4.1.3.

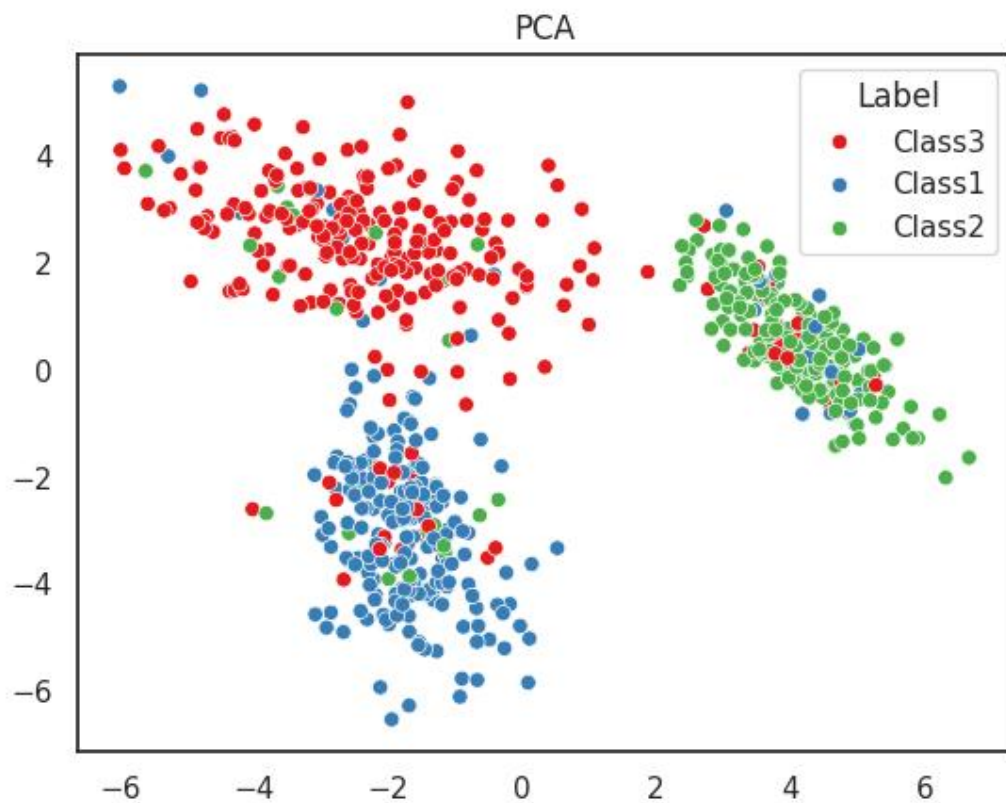


Рисунок 4.1.2 - Диаграмма рассеяния PCA.

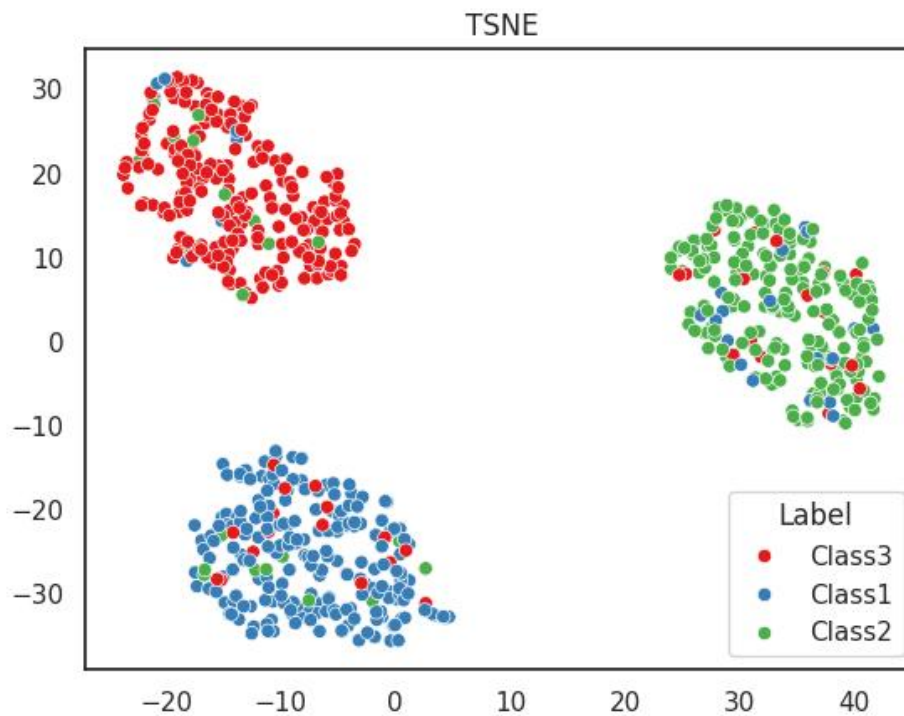


Рисунок 4.1.3 - Диаграмма рассеяния TSNE.

Для сохранения 90% информации в PCA оказалось достаточно 3 компонент из 5, что говорит о наличии корреляций между признаками и возможности сокращения размерности без серьезных потерь. На двумерной проекции PCA классы частично разделяются, особенно Class1 и Class2, однако границы остаются размытыми. TSNE, в отличие от PCA, лучше выделяет локальные структуры: на его графике классы образуют более компактные и отделенные кластеры, но TSNE не сохраняет глобальные расстояния и масштабы, поэтому его результаты интерпретируются только визуально.

Ссылка на полный код находится в Приложении.

Выводы

В ходе выполнения лабораторной работы была проведена предобработка данных. На этапе анализа установлено, что набор из 700 наблюдений не содержит пропусков, признаки имеют различный масштаб и распределение классов неравномерно. Визуализация через PairGrid и гистограммы с KDE показала, что признаки Var1, Var2 и Var5 несут наибольшую информацию для разделения классов, тогда как Var3 и Var4 менее информативны. Классы не являются линейно разделимыми в исходном пространстве.

При преобразовании данных продемонстрированы различия между LabelEncoder и OneHotEncoder: для целевой переменной в задаче классификации достаточно целочисленного кодирования. Сравнение четырех методов масштабирования показало, что StandardScaler является универсальным выбором для данного набора, однако при наличии выбросов предпочтительнее RobustScaler.

Методы понижения размерности подтвердили выводы визуального анализа: PCA позволяет сократить размерность до 3 компонент с сохранением 90% дисперсии, а TSNE дает более наглядное разделение классов в двумерном пространстве.

ПРИЛОЖЕНИЕ

1. Ссылка на код:

https://colab.research.google.com/drive/1VccSA7FIV5I_RWT78_HRzCPZnfX8KbsT?usp=sharing.